Creazione di database e tabelle

SQL

- Ci siamo concentrati sull'esecuzione di query e sulla lettura di dati da database e tabelle esistenti.
- Spostiamo ora la nostra attenzione sulla creazione dei nostri database e tabelle

SQL

- Overview
  - Data Types
  - Primary and Foreign Keys
  - Constraints
  - CREATE
  - INSERT
  - UPDATE
  - DELETE, ALTER, DROP

Data Types

SQL

- Boolean
  - True or False
- Character
  - char, varchar, and text
- Numeric
  - integer and floating-point number
- Temporal
  - date, time, timestamp, and interval

SQL

- UUID
  - Universally Unique Identifiers
- Array
  - Stores an array of strings, numbers, etc.
- JSON
- Hstore key-value pair
- Special types such as network address and geometric data.
-

SQL

Name	Storage Size	Description	Range
`smallint`	2 bytes	small-range integer	-32768 to +32767
`integer`	4 bytes	typical choice for integer	-2147483648 to +2147483647
`bigint`	8 bytes	large-range integer	-9223372036854775808 to +9223372036854775807
`decimal`	variable	user-specified precision, exact	up to 131072 digits before the decimal point; up to 16383 digits after the decimal point
`numeric`	variable	user-specified precision, exact	up to 131072 digits before the decimal point; up to 16383 digits after the decimal point
`real`	4 bytes	variable-precision, inexact	6 decimal digits precision
`double precision`	8 bytes	variable-precision, inexact	15 decimal digits precision
`smallserial`	2 bytes	small autoincrementing integer	1 to 32767
`serial`	4 bytes	autoincrementing integer	1 to 2147483647
`bigserial`	8 bytes	large autoincrementing integer	1 to 9223372036854775807

Primary and Foreign Keys

SQL

- Una chiave primaria è una colonna o un gruppo di colonne utilizzato per identificare una riga in modo univoco in una tabella.

- Ad esempio, nel nostro database dvdrental abbiamo visto che i clienti avevano una colonna *customer_id* **univoca e non nulla** come chiave primaria.

SQL

- Le chiavi primarie sono importanti poiché ci consentono di discernere facilmente quali colonne dovrebbero essere utilizzate per unire le tabelle.

SQL

- Esempio

Query Editor Query History

```
1  SELECT * FROM customer
```

Data Output Explain Messages Notifications

	customer_id [PK] integer	store_id smallint	first_name character varying (45)	last_name character varying (45)
1	524	1	Jared	Ely
2	1	1	Mary	Smith
3	2	1	Patricia	Johnson
4	3	1	Linda	Williams

SQL

- Esempio

Query Editor Query History

```
1  SELECT * FROM customer
```

Data Output Explain Messages Notifications

	customer_id [PK] integer	store_id smallint	first_name character varying (45)	last_name character varying (45)
1	524	1	Jared	Ely
2	1	1	Mary	Smith
3	2	1	Patricia	Johnson
4	3	1	Linda	Williams

SQL

- basata su numeri interi e univoci

Query Editor Query History

```
1  SELECT * FROM customer
```

Data Output Explain Messages Notifications

	customer_id [PK] integer	store_id smallint	first_name character varying (45)	last_name character varying (45)
1	524	1	Jared	Ely
2	1	1	Mary	Smith
3	2	1	Patricia	Johnson
4	3	1	Linda	Williams

SQL

- vedremo SERIAL data type

Query Editor Query History

```
1  SELECT * FROM customer
```

Data Output Explain Messages Notifications

	customer_id [PK] integer	store_id smallint	first_name character varying (45)	last_name character varying (45)
1	524	1	Jared	Ely
2	1	1	Mary	Smith
3	2	1	Patricia	Johnson
4	3	1	Linda	Williams

SQL

- Una chiave esterna è un campo o un gruppo di campi in una tabella che identifica in modo univoco una riga in un'altra tabella.

- Una chiave esterna è definita in una tabella che fa riferimento alla chiave primaria dell'altra tabella.

SQL

- La tabella che contiene la chiave esterna è chiamata tabella figlio.
- La tabella a cui fa riferimento la chiave esterna è chiamata tabella riferita o tabella padre.
- Una tabella può avere più chiavi esterne a seconda delle sue relazioni con altre tabelle.

SQL

- Ricordiamo nella tabella dei pagamenti del database dvdrental, ogni riga di pagamento aveva il suo payment_id univoco (una chiave primaria) e identificava il cliente che ha effettuato il pagamento tramite customer_id (una chiave esterna poiché fa riferimento alla chiave primaria della tabella del cliente)

SQL

- Esempio

Query Editor Query History

```
1  SELECT * FROM payment
```

Data Output Explain Messages Notifications

	payment_id [PK] integer	customer_id smallint	staff_id smallint	rental_id integer	amount numeric (5,2)
1	17503	341	2	1520	7.99
2	17504	341	1	1778	1.99
3	17505	341	1	1849	7.99
4	17506	341	2	2829	2.99

SQL

- Primary Key

Query Editor Query History

```
1  SELECT * FROM payment
```

Data Output Explain Messages Notifications

	payment_id [PK] integer	customer_id smallint	staff_id smallint	rental_id integer	amount numeric (5,2)
1	17503	341	2	1520	7.99
2	17504	341	1	1778	1.99
3	17505	341	1	1849	7.99
4	17506	341	2	2829	2.99

SQL

- Foreign key

Query Editor Query History

```
1  SELECT * FROM payment
```

Data Output Explain Messages Notifications

	payment_id [PK] integer	customer_id smallint	staff_id smallint	rental_id integer	amount numeric (5,2)
1	17503	341	2	1520	7.99
2	17504	341	1	1778	1.99
3	17505	341	1	1849	7.99
4	17506	341	2	2829	2.99

SQL

- La chiave primaria e la chiave esterna in genere sono buoni candidati per unire due o più tabelle.

SQL

- Durante la creazione di tabelle e la definizione di colonne, possiamo utilizzare i vincoli per definire le colonne come chiave primaria o collegare una relazione di chiave esterna a un'altra tabella.

Constraints

SQL

- I vincoli sono le regole applicate alle colonne di dati sulla tabella.
- Questi vengono utilizzati per impedire l'inserimento di dati non validi nel database.
- Ciò garantisce l'accuratezza e l'affidabilità dei dati nel database.

SQL

- I vincoli possono essere suddivisi in due categorie principali:
- Vincoli di colonna
- Vincola i dati in una colonna affinché rispettino determinate condizioni.
- Vincoli di tabella
- applicata all'intera tabella piuttosto che a una singola colonna.

SQL

I vincoli più comuni utilizzati:

NOT NULL Vincolo

Assicura che una colonna non possa avere un valore NULL.

Vincolo UNICO

Assicura che tutti i valori in una colonna siano diversi.

SQL

I vincoli più comuni utilizzati:

PRIMARY KEY

Identifica in modo univoco ogni riga/record in una tabella di database.

FOREIGN KEY

Vincola i dati in base alle colonne in altre tabelle.

SQL

I vincoli più comuni utilizzati:

CHECK Vincolo

Assicura che tutti i valori in una colonna soddisfino determinate condizioni.

SQL

I vincoli più comuni utilizzati:

EXCLUSION Vincolo

Assicura che se due righe qualsiasi vengono confrontate nella colonna o nell'espressione specificata utilizzando l'operatore specificato, non tutti questi confronti restituiranno TRUE.

SQL

- Vincoli di tabella
- CHECK (condizione)
- per verificare una condizione durante l'inserimento o l'aggiornamento dei dati.
- REFERENCES
- per vincolare il valore memorizzato nella colonna che deve esistere in una colonna in un'altra tabella.

SQL

- Vincoli di tabella
- UNIQUE (colonna_elenco)
- forza i valori memorizzati nelle colonne elencate all'interno delle parentesi ad essere univoci.
- PRIMARY KEY(elenco_colonna)
  - Consente di definire la chiave primaria composta da più colonne.

CREATE

SQL

Sintassi generale completa

```
CREATE TABLE nome_tabella (
  nome_colonna TIPO vincolo_colonna,
  nome_colonna TIPO vincolo_colonna,
   tabella_vincolo tabella_vincolo
) INHERITS nome_tabella_esistente;
```

SQL

Sintassi generale completa

```
CREATE TABLE nome_tabella (
  nome_colonna TIPO vincolo_colonna,
  nome_colonna TIPO vincolo_colonna,
   tabella_vincolo tabella_vincolo
) INHERITS nome_tabella_esistente;
```

SQL

Sintassi generale completa

```
CREATE TABLE nome_tabella (
  nome_colonna TIPO vincolo_colonna,
  nome_colonna TIPO vincolo_colonna,
  tabella_vincolo tabella_vincolo
) INHERITS nome_tabella_esistente;
```

SQL

Sintassi generale completa

```
CREATE TABLE nome_tabella (
  nome_colonna TIPO vincolo_colonna,
  nome_colonna TIPO vincolo_colonna,
   tabella_vincolo tabella_vincolo
) INHERITS nome_tabella_esistente;
```

SQL

Sintassi generale completa

```
CREATE TABLE nome_tabella (
  nome_colonna TIPO vincolo_colonna,
  nome_colonna TIPO vincolo_colonna,
  tabella_vincolo tabella_vincolo
) INHERITS nome_tabella_esistente;
```

SQL

Sintassi generale completa

```
CREATE TABLE nome_tabella (
  nome_colonna TIPO vincolo_colonna,
  nome_colonna TIPO vincolo_colonna,
  tabella_vincolo tabella_vincolo
) INHERITS nome_tabella_esistente;
```

SQL

Sintassi generale completa

```
CREATE TABLE nome_tabella (
  nome_colonna TIPO vincolo_colonna,
  nome_colonna TIPO vincolo_colonna,
  tabella_vincolo tabella_vincolo
) INHERITS nome_tabella_esistente;
```

SQL

Sintassi semplice comune

```
CREATE TABLE nome_tabella (
  nome_colonna TIPO vincolo_colonna,
  nome_colonna TIPO vincolo_colonna,
  );
```

SQL

Sintassi di esempio

```
CREATE TABLE nome_tabella (
  nome_colonna TIPO vincolo_colonna,
  nome_colonna TIPO vincolo_colonna,
  );
```

SQL

Sintassi di esempio

```
CREATE TABLE giocatori(
  nome_colonna TIPO vincolo_colonna,
  nome_colonna TIPO vincolo_colonna,
  );
```

SQL

Sintassi di esempio

```
CREATE TABLE giocatori(
  nome_colonna TIPO vincolo_colonna,
  nome_colonna TIPO vincolo_colonna,
  );
```

SQL

Sintassi di esempio

```
CREATE TABLE giocatori(
  nome_colonna TIPO vincolo_colonna,
  nome_colonna TIPO vincolo_colonna,
  );
```

SQL

Sintassi di esempio

```
CREATE TABLE giocatori(
  player_id TYPE column_constraint,
  nome_colonna TIPO vincolo_colonna,
  );
```

SQL

Sintassi di esempio

```
CREATE TABLE giocatori(
  player_id TYPE column_constraint,
  nome_colonna TIPO vincolo_colonna,
  );
```

SQL

- Esempio
  - CREATE TABLE players(
    player_id SERIAL column_constraint,
    column_name TYPE column_constraint,
    );

SQL

- SERIAL
  - In PostgreSQL, una sequenza è un tipo speciale di oggetto di database che genera una sequenza di numeri interi.
  - Una sequenza viene spesso utilizzata come colonna chiave primaria in una tabella.

SQL

- SERIAL
  - Creerà un oggetto sequenza e imposterà il valore successivo generato dalla sequenza come valore predefinito per la colonna.
  - Questo è perfetto per una chiave primaria, perché registra automaticamente voci intere univoche al momento dell'inserimento.

SQL

- SERIAL
  - Se una riga viene successivamente rimossa, la colonna con il tipo di dati SERIAL non verrà regolata, segnalando il fatto che una riga è stata rimossa dalla sequenza,
  - ad esempio 1,2,3,5,6,7
    - Sai che la riga 4 è stata rimossa a un certo punto

SQL

Name	Storage Size	Description	Range
`smallint`	2 bytes	small-range integer	-32768 to +32767
`integer`	4 bytes	typical choice for integer	-2147483648 to +2147483647
`bigint`	8 bytes	large-range integer	-9223372036854775808 to +9223372036854775807
`decimal`	variable	user-specified precision, exact	up to 131072 digits before the decimal point; up to 16383 digits after the decimal point
`numeric`	variable	user-specified precision, exact	up to 131072 digits before the decimal point; up to 16383 digits after the decimal point
`real`	4 bytes	variable-precision, inexact	6 decimal digits precision
`double precision`	8 bytes	variable-precision, inexact	15 decimal digits precision
`smallserial`	2 bytes	small autoincrementing integer	1 to 32767
`serial`	4 bytes	autoincrementing integer	1 to 2147483647
`bigserial`	8 bytes	large autoincrementing integer	1 to 9223372036854775807

SQL

Name	Storage Size	Description	Range
`smallint`	2 bytes	small-range integer	-32768 to +32767
`integer`	4 bytes	typical choice for integer	-2147483648 to +2147483647
`bigint`	8 bytes	large-range integer	-9223372036854775808 to +9223372036854775807
`decimal`	variable	user-specified precision, exact	up to 131072 digits before the decimal point; up to 16383 digits after the decimal point
`numeric`	variable	user-specified precision, exact	up to 131072 digits before the decimal point; up to 16383 digits after the decimal point
`real`	4 bytes	variable-precision, inexact	6 decimal digits precision
`double precision`	8 bytes	variable-precision, inexact	15 decimal digits precision
`smallserial`	2 bytes	small autoincrementing integer	1 to 32767
`serial`	4 bytes	autoincrementing integer	1 to 2147483647
`bigserial`	8 bytes	large autoincrementing integer	1 to 9223372036854775807

SQL

- Example Syntax
  - CREATE TABLE players(
    player_id SERIAL column_constraint,
    column_name TYPE column_constraint,
    );

SQL

- Example Syntax
  - CREATE TABLE players(
    player_id SERIAL column_constraint,
    column_name TYPE column_constraint,
    );

SQL

- Example Syntax
  - CREATE TABLE players(
    player_id SERIAL PRIMARY KEY,
    column_name TYPE column_constraint,
    );

SQL

- Example Syntax
  - CREATE TABLE players(
    player_id SERIAL PRIMARY KEY,
    column_name TYPE column_constraint,
    );

SQL

- Example Syntax
  - CREATE TABLE players(
    player_id SERIAL PRIMARY KEY,
    age TYPE column_constraint,
    );

SQL

- Example Syntax
  - CREATE TABLE players(
    player_id SERIAL PRIMARY KEY,
    age TYPE column_constraint,
    );

SQL

Name	Storage Size	Description	Range
`smallint`	2 bytes	small-range integer	-32768 to +32767
`integer`	4 bytes	typical choice for integer	-2147483648 to +2147483647
`bigint`	8 bytes	large-range integer	-9223372036854775808 to +9223372036854775807
`decimal`	variable	user-specified precision, exact	up to 131072 digits before the decimal point; up to 16383 digits after the decimal point
`numeric`	variable	user-specified precision, exact	up to 131072 digits before the decimal point; up to 16383 digits after the decimal point
`real`	4 bytes	variable-precision, inexact	6 decimal digits precision
`double precision`	8 bytes	variable-precision, inexact	15 decimal digits precision
`smallserial`	2 bytes	small autoincrementing integer	1 to 32767
`serial`	4 bytes	autoincrementing integer	1 to 2147483647
`bigserial`	8 bytes	large autoincrementing integer	1 to 9223372036854775807

SQL

Name	Storage Size	Description	Range
`smallint`	2 bytes	small-range integer	-32768 to +32767
`integer`	4 bytes	typical choice for integer	-2147483648 to +2147483647
`bigint`	8 bytes	large-range integer	-9223372036854775808 to +9223372036854775807
`decimal`	variable	user-specified precision, exact	up to 131072 digits before the decimal point; up to 16383 digits after the decimal point
`numeric`	variable	user-specified precision, exact	up to 131072 digits before the decimal point; up to 16383 digits after the decimal point
`real`	4 bytes	variable-precision, inexact	6 decimal digits precision
`double precision`	8 bytes	variable-precision, inexact	15 decimal digits precision
`smallserial`	2 bytes	small autoincrementing integer	1 to 32767
`serial`	4 bytes	autoincrementing integer	1 to 2147483647
`bigserial`	8 bytes	large autoincrementing integer	1 to 9223372036854775807

SQL

- Example Syntax
  - CREATE TABLE players(
    player_id SERIAL PRIMARY KEY,
    age TYPE column_constraint
    );

SQL

- Example Syntax
  - CREATE TABLE players(
    player_id SERIAL PRIMARY KEY,
    age SMALLINT column_constraint
    );

SQL

- Example Syntax
  - CREATE TABLE players(
    player_id SERIAL PRIMARY KEY,
    age SMALLINT column_constraint
    );

SQL

- Example Syntax
  - CREATE TABLE players(
    player_id SERIAL PRIMARY KEY,
    age SMALLINT NOT NULL
    );

SQL

- Example Syntax
  - CREATE TABLE players(
    player_id SERIAL PRIMARY KEY,
    age SMALLINT NOT NULL
    );

INSERT

SQL

- INSERT consenti l’inserimento di righe in tabella.
- Sintassi
  - INSERT INTO table (column1, column2, ...)
    VALUES
    (value1, value2, ...),
    (value1, value2, ...) ,...;

SQL

- INSERT consente di aggiungere righe a una tabella.
- Sintassi per l'inserimento di valori da un'altra tabella:
  - INSERT INTO tabella(colonna1,colonna2,...)
    - SELECT colonna1,colonna2,...
    - FROM un'altra_tabella
    - WHERE condizione;

SQL

- Tieni presente che i valori di riga inseriti devono corrispondere per la tabella, inclusi i vincoli.

UPDATE

SQL

- La parola chiave UPDATE consente di modificare i valori delle colonne in una tabella.

SQL

- General Syntax
  - UPDATE table
    SET column1 = value1,
    column2 = value2 ,...
    WHERE
    condition;

SQL

- Example
  - UPDATE account
    SET last_login = CURRENT_TIMESTAMP
    WHERE last_login IS NULL;

SQL

- Reset senza WHERE condition
  - UPDATE account
    SET last_login = CURRENT_TIMESTAMP

SQL

- Set sulla base del valore di un’altra colonna
  - UPDATE account
    SET last_login = created_on

SQL

- UPDATE join
  - UPDATE TableA
    SET original_col = TableB.new_col
    FROM tableB
    WHERE tableA.id = TableB.id

SQL

- Ritornare le righe modificate
  - UPDATE account
    SET last_login = created_on
    RETURNING account_id,last_login

DELETE

SQL

- Possiamo usare la clausola DELETE per rimuovere righe da una tabella
- Esempio:
  - DELETE FROM table
    WHERE row_id = 1

SQL

- Possiamo eliminare le righe in base alla loro presenza in altre tabelle
- For example:
  - DELETE FROM tableA
    USING tableB
    WHERE tableA.id=TableB.id

SQL

- Cancellazione di tutte le righe
- For example:
  - DELETE FROM table

SQL

- Analogamente al comando UPDATE, puoi anche aggiungere una chiamata RETURNING per restituire le righe che sono state rimosse.

ALTER

SQL

- La clausola ALTER consente modifiche a una struttura di tabella esistente, ad esempio:
  - Aggiunta, eliminazione o ridenominazione di colonne
  - Modifica del tipo di dati di una colonna
  - Impostare i valori DEFAULT per una colonna
  - Aggiungere vincoli CHECK
  - Rinomina tabella

SQL

- Sintassi generale
  - ALTER TABLE table_name action

SQL

- Aggiunta di colonne
  - ALTER TABLE table_name

    ADD COLUMN new_col TYPE

SQL

- Rimozione di colonne
  - ALTER TABLE table_name

    DROP COLUMN col_name

SQL

- Modifica di vincoli
  - ALTER TABLE table_name

    ALTER COLUMN col_name

    SET DEFAULT value

SQL

- Modifica di vincoli
  - ALTER TABLE table_name

    ALTER COLUMN col_name

    DROP DEFAULT

SQL

- Modifica vincoli
  - ALTER TABLE table_name

    ALTER COLUMN col_name

    SET NOT NULL

SQL

- Modifica vincoli
  - ALTER TABLE table_name

    ALTER COLUMN col_name

    DROP NOT NULL

SQL

- Modifica vincoli
  - ALTER TABLE table_name

    ALTER COLUMN col_name

    ADD CONSTRAINT constraint_name

DROP

SQL

- DROP consente la rimozione completa di una colonna in una tabella.
- In PostgreSQL questo rimuoverà automaticamente anche tutti i suoi indici e vincoli che coinvolgono la colonna.
- Tuttavia, non rimuoverà le colonne utilizzate nelle viste, nei trigger o nelle stored procedure senza la clausola CASCADE aggiuntiva.

SQL

- Sintassi generale
  - ALTER TABLE table_name

    DROP COLUMN col_name

SQL

- Rimozione delle dipendenze
  - ALTER TABLE table_name

    DROP COLUMN col_name CASCADE

SQL

- Verifica dell’esistenza
  - ALTER TABLE table_name

    DROP COLUMN IF EXISTS col_name

SQL

- Drop colonne multiple
  - ALTER TABLE table_name

    DROP COLUMN col_one,

    DROP COLUMN col_two

CHECK

SQL

- Il vincolo CHECK ci consente di creare vincoli più personalizzati che aderiscono a una determinata condizione.
- Come assicurarsi che tutti i valori interi inseriti scendano al di sotto di una certa soglia.

SQL

- Sintasi generale
  - CREATE TABLE example(
    ex_id SERIAL PRIMARY KEY,
    age SMALLINT CHECK (age > 21),
    parent_age SMALLINT CHECK (
    parent_age > age)
    );